REFERENCE · GENALPHA-BSS

The Operator's Manual

How to run, configure and operate the BSS — by role and by task. The companion volume, The Honest Machine (a build memoir · verify everything — how one person and an AI built a complete telecom suite and proved every piece of it), tells how it was built and why; this book tells you which button, which endpoint, which environment variable. Everything here is proven by the suite named beside it.

Contents

1 · Getting started

```
docker compose up -d          # ~90 containers; first build takes a while
open http://localhost:8080/shop/   # the storefront (guest browsing works)
```

The gateway is `:8080`; Keycloak is `:8085` (admin/admin). Two seed operators run side by side: **genalpha** (EN/EUR, realm `bss`) and **nova** (NO/NOK, realm `nova`) — plus any operator you mint (§7). Demo card `4242 4242 4242 4242` pays; anything ending `0002` declines. Promo code `WELCOME10`. Serviceable fiber postcodes start `111` / `222` / `333`.

> **The one discipline:** every feature in this manual has an end-to-end Playwright suite in `ops/e2e/`. When in doubt about how something behaves, the suite is the specification: `node ops/e2e/<name>_test.js` runs it against the live stack.

2 · Surfaces & sign-ins

URL	WHO	DEMO SIGN-IN
`/shop/`	Customers (self-register or guest)	`kai@bss.local` / `kai`
`/app/`	Customers on mobile (Expo web)	`emil@acme.example` / `emil`
`/biz/`	B2B org admins + invited members	`bianca@acme.example` / `bianca`
`/csr/`	Agents (role-scoped powers)	`agent-anna` / `agent` ; junior: `jo@bss.local` / `jo`
`/console/`	Back office (role-scoped tabs)	`demo` / `demo` ; product-only: `pat@bss.local` / `pat`
`/dealer-app/`	Retail-partner clerks & telesales agents	any account whose org holds a dealer agreement
`shop.nova.localhost:8080/shop/`	Nova's white-label storefront	`nils@nova.local` / `nils`
`biz.nova.localhost:8080/biz/`	Nova's B2B console (Norwegian)	`birgit@fjellheim.no` / `birgit`

White-labeling is hostname-driven: `shop.<tenant>.localhost` serves that operator's brand, locale and currency from the live registry (§7).

3 · The product owner

Catalog (console → Product Offerings / Specifications / Prices)

Offerings are TMF620 data: hard and soft bundles (`bundledProductOfferingOption` cardinality, pick-N-of-M choice groups), characteristic-conditioned prices ("+2.00/mo when colour = Titanium"), commitment terms, categories that drive placement *and* fulfilment (Top-ups and Insurance are billing-only; Partner services mint entitlement codes; everything else draws an MSISDN + SIM). Artwork rides the offering's attachments — internal store or the tenant's own PIM; the internal store's bytes live in-row by default or in S3-protocol / Azure Blob object storage (deployment config, §9 — suite #54 proves all three).

Rules & pricing as data (console → Rules)

Order rules and dynamic pricing are JSON-logic rows with a dry-run panel. A price change is a rule, not a release. See `docs/product-rules.md` for the authoring guide.

Product Copilot (console → Copilot)

Chat a product into the catalog: the model proposes specs/prices/offerings as TMF620 payloads, the console shows a human card, and a deterministic executor applies it with *your* token on confirmation. The model never writes. The copilot also authors **pricing rules** ("10% off the phone with this plan") and — suite #61 — **personalization experience rules** ("device browsers see this banner and this pin"): both become policy rows you can disable in Rules.

Product Advisor (console → Product advisor) — suite #52

Findings with receipts: top-up attach counted from inventory ("2 of 2 customers on this plan also bought top-ups"), market price gaps from the tenant's market feed (10% materiality threshold — small gaps are deliberately silent), an LLM narrative that never invents a number. **Adopt as draft…** births an `In study` offering; nothing reaches the shop until you promote it.

Campaigns & growth (console → Campaigns / Journeys / Audiences)

Segments form from consented shop behavior; campaigns prove lift against real holdouts; journeys read the customer before speaking; A/B arms split deterministically; guardrails (frequency caps, quiet hours) are settings. Every message lands on the TMF683 timeline.

4 · The CSR

Search the 360 (multi-word, name-ranked, with a trigram typo net — "Solvieg" finds Solveig when strict matching finds nothing), then act — every action logs itself on the customer's timeline:

THE CUSTOMER ASKS	THE BUTTON / ACT
"What's my PUK?" / "reset my PIN"	Reveal PUK · Reset PIN (owner notified — a silent credential change reads as fraud)
"I lost my SIM"	Replace SIM (old card quarantined first)
"new number, please"	Change number (old number quarantined with its story)
"pause my subscription"	Pause / Resume (charging pauses at the OCS; auto-resumes on the promised date)
"I can't pay all of it"	Installments (parts sum exactly; miss one → reminder → grace → dunning)
"this charge is wrong"	Dispute (collection pauses; resolution credits or upholds, in writing)
"send me my invoice"	PDF (the exact document) · Email bill (to the address on file, never one dictated on the phone)
"my internet is slow"	Diagnose (triage across usage/assurance/network facts)
"give my number to my colleague"	Transfer (three-way authorization; B2B handover keeps the company paying)

Self-care mirrors most of these in the shop and app — customers act on their OWN line (ownership-checked; a foreign line answers 404, never 403).

The knowledge base at the desk

Articles are audience-shelved (customer / CSR / sales / product owner) and the CSR's ask-with-sources grounds its answers in them. Search is full-text, ranked, and stems in **the tenant's own language** — the locale that picks the storefront's currency also picks the stemmer, so on nova "regning" finds "regningene" (suite #55). Typos in customer search are caught by a trigram net that only speaks when strict matching finds nothing — strict results are byte-identical to before. Beneath the keyword search sits the **semantic net** (pgvector): "why is my internet so slow" finds the fair-use article that contains neither word, and nonsense finds nothing — the cosine ceiling keeps the net honest. Embeddings are a seam: `bss.knowledge.embeddings` = `stub` (deterministic, keyless) or `openai-compatible` (any real model).

5 · Billing & money operations

The billing run

`POST /tmf-api/customerBillManagement/v4/billingRun` (staff). Segment-based and per-account: mid-month starts, mid-cycle plan changes and ceases each bill exactly their own days, labelled ("from the 16th, 15 days"). Payday alignment per customer (`POST /individual/{id}/billingCycle` , day 1–28). The invariant the suites enforce from every direction: **no day is ever billed twice**.

The bill as a document — suite #46

`GET /customerBill/{id}/document.pdf` (owner-scoped); `POST /customerBill/{id}/resend` emails it to the address on file. Delivery preference per customer: `paper | einvoice | digital` (`POST /individual/{id}/billDelivery`) — the customer picks the channel; the tenant keeps the plumbing.

E-invoice formats are rows (console → Bill formats)

CODE	STANDARD	SYNTAX
`ehf`	EHF Billing 3.0 (Norway; carries the KID)	UBL 2.1
`peppol`	Peppol BIS Billing 3.0	UBL 2.1
`cii`	EN 16931	UN/CEFACT CII
`aunz`	A-NZ Peppol BIS 3.0	UBL 2.1
`edifact`	UN/EDIFACT INVOIC D96A	segments
`facturx`	Factur-X / ZUGFeRD	PDF + embedded CII

Adding a country is an INSERT (the suite added XRechnung through the form). The tenant's `BILL_DISTRIBUTION_FORMAT` picks a row by code; the renderer follows the row.

Delivery, status, and the money coming home — suites #46/#47

Deliveries (console tab): the outbox-backed ledger — every bill's trip, attempts, the buyer's Peppol Invoice Response (accepted / rejected with their words / paid), one-click Retry on failures. **Remittance**: the bank posts camt.054, Nets OCR or BAI2 to `POST /bank/v1/remittance` (per-tenant `BANK_TOKEN`); a matched KID settles the bill through the card path's own guarantee; everything unclear parks on **Unapplied cash** (console tab) with its reason. Money is fail-closed: never guessed, never dropped.

Dunning (console → Dunning)

The collections worklist: overdue installments, broken plans, what is still owed.

6 · Partner channels — suites #48/#51

> **Being a dealer is a row.** Sign a chain: `POST /dealer/v1/agreement` (staff) with the org, commission per activation, and — for a chain with its own POS — the OAuth2 `clientId` that speaks for them. Clerks are org membership; foreign dealers see 404-shaped nothing.

Retail (the CSP + external-retail model — think Elkjøp/Power)

Counter sale: sell against the customer's email; the order carries the dealer stamp. **Starter kits**: mint batches (code + SIM + attribution in the box); the customer self-activates (`POST /dealer/v1/starterKit/activate`) and the line rides the SIM from the box. **POS API**: same endpoints, machine credential, per-partner rate limits at the edge (429 + Retry-After); the chain's own phone rides the sale as `device` context — never a billable item. **Commission**: pending → earned after the angrerett window → clawed back with the reason if the customer leaves inside it.

Telesales (outbound, shaped by the law)

Dial list: `GET /dealer/v1/telesales/dialList?segment=` — consented at the source (insight segments), every number DNC-washed, reserved citizens excluded *and counted*. **The call makes an OFFER** (`POST /dealer/v1/telesales/offer`): washed fail-closed (reserved refuses, unreachable register refuses, unconfigured tenant refuses); no order exists yet. **The customer confirms in writing** (`POST /telesales/v1/confirm` , signed in, code from their inbox — or, for a cold prospect, after registering with the offered email); only then the order, the activation and the commission are born. Silence expires on a clock. Every dialer call logs to TMF683.

7 · The host administrator

The tenant fleet is a file

`infra/tenants/tenants.yml` is the shared registry (mounted into all 31 services); every service live-refreshes it. NEW operators join the running fleet within one refresh interval; changes to a serving operator's seams/brand/hosts apply live; identity fields (`id` , `issuer` , key endpoints) require a restart — by design.

Minting an operator — suites #49/#50

```
# the script (~2 minutes to a billed customer):
ops/onboard-tenant.sh fjord "Fjord Mobil" da DKK "#1B6CA8"

# or the form: console → Operators → five fields → Save
#   (host-tenant admins only; realm clone + registry block + seeded catalog;
#    shop.<id>.localhost wears the brand the moment the gateway refreshes)
# live rebrand of a form-born operator:
PATCH /onboarding/v1/operator/{id}   {"name","color","locale","currency"}
```

Seed operators (genalpha, nova) are protected from the form — their config is env.

Staff & roles (console → Staff)

Grant/revoke whole areas per operator over the tenant's own IdP (TMF672) — no Keycloak console needed for daily administration.

8 · AI configuration & governance — suites #53/#59

The individualized shop (suite #60)

`GET /ai/v1/forYou` — the signed-in customer's own rail (self-scoped: party = token subject, no parameter to probe). TMF680 candidates + consented interests + churn-risk loyalty flag; ONE governed FAST caption per 5-minute cache window ("Picked for you after your look at devices"). No personalization consent → zero interests, generic caption. The storefront renders it as the "Recommended for you" shelf, and the mobile app's For-you card rides the same endpoint (same cache — the second channel is free).

The control plane (suite #59)

Every LLM turn is **metered** (token counts + cost land on the AI Audit ledger — the console's AI Audit tab shows tokens, cost and outcome per turn) and **governed** per tenant: `GET /ai/v1/governance` shows spend this window, the ceiling, remaining headroom and the **agent-action trail** (which AI wrote which resource — e.g. the advisor's `catalog.createDraftOffering`). `POST /ai/v1/governance/budget {"budgetMicros": 5000000, "windowHours": 720, "enabled": true}` sets the ceiling and the **kill-switch**; crossing the ceiling refuses fail-closed (429, audited as `refused-budget`), `enabled=false` refuses instantly (403). No budget row = unlimited-and-enabled — governance is opt-in per tenant. Cost is estimated (chars/4 tokens × per-model rate, `bss.ai.prices.<model>` in micros per 1K, default `bss.ai.default-price-per-1k-micros`); exact provider usage is a wired-later seam.

Per tenant, all data: `ai-provider` (`stub` — keyless CI default — `openai-compatible`, `anthropic`), `ai-base-url`, `ai-api-key`, `ai-model`. **Tiered routing**: the task class lives at the call site — FAST (copywriting, summaries, narratives) vs SMART (product proposals, next-best-offer; the default when unclassified). Every value resolves tier-first:

```
AI_MODEL_FAST=swift-mini            # cheap model for volume work
AI_MODEL_SMART=frontier-x           # careful model for judgment
AI_PROVIDER_SMART=anthropic         # tiers go down to the PROVIDER:
AI_BASE_URL_SMART=https://api...    # a local endpoint for volume,
AI_API_KEY_SMART=...                # a frontier API for judgment — at once
```

9 · The seam catalog

Every external system is a per-tenant seam: configure it and the feature lights up; leave it blank and the feature degrades honestly (fail-open for enrichment, fail-closed for money, identity and consent).

SEAM	TENANT FIELDS (ENV)	BEHAVIOR WHEN BLANK
Email provider (ESP)	`DELIVERY_PROVIDER, ESP_URL, ESP_API_KEY, ESP_FROM`	in-app inbox only
Bill distribution partner	`BILL_DISTRIBUTION_{PROVIDER,URL,TOKEN,FORMAT,CHANNEL}`	bills stay in-app/PDF
Bank (remittance)	`BANK_TOKEN`	webhook refuses (money: fail-closed)
Do-not-call register	`DNC_URL, DNC_TOKEN`	outbound telesales refuses (opt-IN by configuring)
Market-intelligence feed	`MARKET_FEED_URL, MARKET_FEED_TOKEN`	advisor's market findings absent; own-data findings stand
LLM	`AI_*` (§8)	keyless stub answers
PIM (product imagery)	`PIM_BASE_URL`	internal TMF667 store
Content store (deployment-level)	`CONTENT_STORE` = `in-row | s3 | azure-blob` (+ `S3_*` / `AZURE_*`)	bytes in the document row (dev-simple)
Embeddings (deployment-level)	`bss.knowledge.embeddings` = `stub | openai-compatible` (+ `embeddings-url/key/model`)	deterministic keyless stub — the semantic net still works, with curated synonyms instead of a model
Web analytics (GA4)	`ANALYTICS_*`	internal insight only

Social platform	`SOCIAL_API_URL, SOCIAL_ACCESS_TOKEN`	no audience push / lead import

Per-tenant variants ride suffixes: `_NOVA`, `_FJORD`, … matching the placeholders in `tenants.yml`.

10 · Environment reference (dev clocks)

VARIABLE	MEANING	DEV / PROD DEFAULT
`BSS_BILLING_DUNNING_TICK_MS / _GRACE_MS`	collections sweep / grace	5s & 20s / 60s & 7d
`BSS_BILLING_DISTRIBUTION_TICK_MS / _RETRY_SECONDS`	delivery relay / backoff base	3s & 4s / 30s & 60s
`BSS_SOM_COMMISSION_WINDOW_MS / _TICK_MS`	angrerett window / harden tick	15s & 5s / 14d & 1h
`BSS_SOM_TELESALES_EXPIRY_MS / _TICK_MS`	offer expiry / sweep	15s & 5s / 48h & 1h
`BSS_TENANTS_REFRESH_MS`	live registry refresh	15s
`BSS_GATEWAY_PARTNER_RATE_CAPACITY / _WINDOW_MS`	per-partner rate limit (dealer surface)	10 per 2s / 60 per 60s
`GLOBAL_RATE_CAPACITY / GLOBAL_RATE_WINDOW_MS`	the wide ring — a ceiling on EVERY path, per subject/client/IP	1200 per 60s

11 · Extension API reference (beyond the TMF standard surfaces)

ENDPOINT	WHO	DOES
`GET /customerBill/{id}/document.pdf`	owner/staff	the bill as a PDF
`POST /customerBill/{id}/resend`	owner/staff	email the invoice to the address on file
`POST /customerBill/{id}/dispute · /installmentPlan`	owner/agent	contest / split a bill
`GET/PATCH/POST /billFormatProfile[/{code}]`	read: billing · write: billing:admin	e-invoice profiles as rows
`GET /billDistribution · POST /billDistribution/{id}/retry`	billing:admin	the delivery ledger
`POST /bank/v1/remittance`	the bank (X-Bank-Token)	camt.054 / OCR / BAI2 in
`POST /distribution/v1/response`	the partner (X-Distribution-Token)	Peppol Invoice Response onto the ledger
`GET /remittance/unapplied`	billing:admin	unapplied cash worklist
`POST /individual/{id}/billingCycle · /billDelivery`	self/staff	payday alignment · delivery preference
`/dealer/v1/*` (agreement, kits, sell, commission, orders/{id}, telesales/*)	staff / dealer / partner POS	the whole dealer & telesales channel
`POST /telesales/v1/confirm`	the customer, signed in	the written yes that births the order
`GET /advisor/v1/findings · POST /advisor/v1/adopt`	catalog:write	product advisor · adopt-as-draft
`GET/POST /onboarding/v1/operator · PATCH …/{id}`	host roles:admin	mint / live-mutate an operator
`GET /app/tenant-config.json`	public, by Host	the white-label manifest

12 · The mock integrations (dev stand-ins)

CONTAINER	PORT	STANDS IN FOR	CHAOS / INSPECTION
mock-esp	8121	SendGrid-shaped email provider	`GET /mails`, webhook receipts
mock-social	8122	Meta-shaped audiences/leads	—
mock-distribution	8124	Peppol access point / print house (validates EHF/EDIFACT/Factur-X!)	`GET /invoices`, `POST /outage`, `POST /invoices/{billNo}/respond`
mock-dnc	8125	reservation register	numbers ending 9999 are reserved; `POST /outage`
mock-market	8126	tariff-comparison feed	`GET /offers`
minio	9000	S3-protocol object store (AWS/R2/on-prem)	—
azurite	10000	Azure Blob Storage (Microsoft's emulator)	—
mock-llm	8127	OpenAI- and Anthropic-dialect LLM	names the model in answers; `GET /requests`

13 · Production operations — suites #56/#57

Ticks that never double-fire

Every mutating scheduled job — dunning, bill delivery, campaign journeys, churn sweeps, porting cutover, commission hardening, order resume, telesales expiry, cart abandonment — claims a row-lease (`tick_lock` table in the service's own database) before it runs. Scale any service to N replicas and each tick still fires once; a crashed holder's lease expires on its own. Inspect a service's leases with `psql -d billing -c "SELECT * FROM tick_lock"`. Suite #56 proves it by running TWO billing replicas and counting exactly one delivered bill at the receiving partner.

The billing run — partitioned, resumable, and on a ledger

Each account's bill commits in its *own* transaction; a failed account is recorded and skipped, never the run. A crashed run RESUMES on the next trigger — every bill already cut is its own checkpoint — and every run is a row: `GET /tmf-api/customerBillManagement/v4/billingRun`

lists recent runs (status running/completed/superseded, accounts, bills, failures, last error). One run speaks at a time: a trigger during a live run answers `{"busy": true}` (the lease heartbeats per account, so a crashed run frees it within ~2 minutes), and the database enforces one bill per account per period with a unique index — constraint, not convention. Suite #57 kills billing mid-run and proves exactly-one-bill-per-customer after the resume.
`BSS_BILLING_RUN_ACCOUNT_DELAY_MS` (default 0) is the dev pacing knob the suite uses to hold a run open.

Load numbers and the smoke SLO

`node ops/load/loadtest.js --seconds 15 --workers 10` — plain-Node closed-loop driver, three scenarios (anonymous catalog, storefront page, authenticated bill reads), reporting req/s and p50/p95/p99. LIFT THE WIDE RING FIRST (`GLOBAL_RATE_CAPACITY=1000000 docker compose up -d gateway`) or the P0 ceiling will throttle the harness — that is it working.
Baselines and their caveats: docs/perf-baselines.md.

Alerts

`infra/prometheus/alert-rules.yml` : ServiceDown (2 min unscraped), OutboxPublishFailures (events queuing, not lost), GatewayServerErrors (5xx > 5%). The whole fleet is scraped (32 targets). View at `:9090/alerts` ; a deployment routes them to a pager via Alertmanager.

Backup and the restore drill

`ops/backup.sh` dumps every database and role to `backups/` (git-ignored, newest 14 kept).
`ops/restore-drill.sh` restores the newest dump into a throwaway container, verifies databases and rows, optionally proves a sentinel (`--expect <db> "<sql>"`), and removes the container. Run the drill on a schedule — an untested backup is a hope, not a backup.

Rate limits, two rings — buckets in Redis

The dealer surface keeps strict per-partner buckets; every other path passes a generous fleet-wide ceiling (per subject for people, per client for machines, per IP for anonymous), answering `429 + Retry-After` when tripped. Dials in §10. The buckets live in Redis (`BSS_GATEWAY_REDIS_URL` ; blank = in-memory): replicas share ONE exact ceiling, a restarted gateway still refuses inside the same window, and an unreachable Redis fails open — fairness, never authorization.

The live k8s soak (on demand) — local, AWS, and Azure

The Helm chart has run on three live clusters: local k3s, AWS EKS, and Azure AKS — each with billing at 2 replicas holding ONE set of tick leases, requests served through the in-cluster

gateway. The cloud runs are one command each: `ops/k8s-soak/eks-run.sh up|smoke|down` and `ops/k8s-soak/aks-run.sh up|smoke|down` (compose must stop first — one laptop, two worlds). Both wrap Terraform + registry push + managed-Postgres + Helm + smoke + teardown. The cloud-by-cloud differences (all absorbed by seams — the app never changed) are tabulated in architecture.md §5; the receipts and the live-run truths each cloud taught are in k8s-soak-plan.md. Smoke: `ops/k8s-soak/smoke.js` against port-forwarded gateway/keycloak.

Managed-Postgres note: point `config.dbHost` at the managed server and set `local.postgres.enabled=false`. Some tiers cap connections low (Azure B1ms ≈ 50) — the chart now holds only one idle connection per pool; raise the server's `max_connections` for headroom. Managed Postgres may also gate extensions (Azure needs pg_trgm/vector allow-listed) and demand TLS (add `sslmode=require` to the datasource URL, or relax on the server for a soak).

The secret gate

`ops/install-hooks.sh` wires `ops/scan-secrets.sh` as a pre-commit hook: any staged diff carrying a real-secret shape (Anthropic/OpenAI/AWS/GitHub/Slack keys, private-key blocks) refuses to commit. Injectable values are listed in `.env.example`; what a real deployment must add beyond this stack — external secrets, TLS in transit, managed HA Postgres/Kafka — is the runbook, docs/hardening.md.

14 · Privacy & GDPR operations — suite #58

The data passport (right of access)

`GET /privacy/v1/export` — a customer exports THEMSELVES with their own sign-in (no role needed; the fan-out rides their token to every service). The DPO — any user with `roles:admin` — adds `?partyId=` to export another person. The JSON names its categories and also the legal-hold categories it cannot delete.

Erasure (right to be forgotten)

`POST /privacy/v1/erase {"partyId": "..."}` (DPO only). Refuses 409 while the person holds ACTIVE services — terminate first. Deletes interactions, messages, marketing, carts, tickets, appointments, behavioural data; anonymizes the profile in place; disables and scrubs the login at the IdP. Bills/payments/orders/usage/agreements are retained with their legal basis named in the report. A partial fan-out answers `"status": "partial — re-run required"` — re-run until `completed`. `GET /privacy/v1/erasure` lists the audit trail.

Retention clocks

`BSS_RETENTION_INTERACTIONS_SECONDS` (party-interaction) and `BSS_RETENTION_TICKETS_SECONDS` (trouble-ticket, settled tickets only) — 0 = keep forever (default). Production sets day-scale values (e.g. 63072000 = 2 years); `BSS_RETENTION_SWEEP_MS` paces the sweep. TickGuard-guarded: replicas never double-sweep.

PCI & lawful intercept, honestly

Card data never enters the BSS — the vault stores `pspToken/brand/lastFour/expiry` (SAQ-A-shaped; a QSA attests the rest). Lawful intercept: the BSS half is warrant-gated subscriber disclosure — shaped, not built; see docs/privacy.md.

15 · The agent channel (agentic commerce) — suite #64

> **Being shopped by AI agents is opt-in.** Every operator carries `agent-commerce: off | discovery | full` in the tenant registry — `off` (dark to agents, 404), `discovery` (the product feed answers; agent checkout refused with 403 — findable, checkout stays human), `full` (delegated checkout too). Newborn operators are born `off`. Flip it on the operator form (console → Operators → Agent commerce) or by env (`AGENT_COMMERCE` , `AGENT_COMMERCE_<ID>`); the gateway enforces it live — no restart.

The surface (ACP, spec 2026-04-17)

Feed: `GET /acp/product_feed` — public projection of the active TMF620 catalog (unconditioned prices only, one-time preferred; unpriced offerings are skipped). **Checkout**: `POST /acp/checkout_sessions` with `{items:[{id,quantity}]}` , then `GET/POST /acp/checkout_sessions/{id}` , `POST …/{id}/complete` (needs `Authorization` + `Idempotency-Key` + `payment_data.token`), `POST …/{id}/cancel` . A session rides a real TMF663 cart; complete charges the payment token and places a TMF622 order marked `category=agenticCommerce` with the buyer as relatedParty. A replayed Idempotency-Key returns the *same* order; a fresh key on a completed session is 409.

Delegated checkout (the trust model)

Agents never hold a machine identity. The shopper's token is exchanged (OAuth token exchange, Keycloak ≥26.2) through the confidential `bss-agent` client — `fullScopeAllowed` off, scope-mapped to exactly `catalog:read ordering:read/write payment:read/write` — so the agent's credential can order and pay and nothing else. The shopper's client must carry an audience

mapper naming `bss-agent`: the realm grants the delegation path explicitly, or the exchange is refused. Suite #64 decodes both tokens to prove the downscoping.

MCP tools (Claude-class agents)

`integrations/mcp-server` wears the same surface as five retail tools: `search_offerings`, `get_offering` (with "also bought"), `start_checkout`, `update_checkout`, `complete_checkout`. Shopper env: `BSS_SHOPPER_USERNAME/PASSWORD`; exchange client: `BSS_AGENT_CLIENT_ID/SECRET`.

Honest scope: the merchant surface is implemented and proven. Appearing *inside* ChatGPT/Perplexity additionally requires the operator's business onboarding with those platforms (feed registration, PSP delegated-payment agreement) — an operator act, not a code gap. The endpoints they register are the ones suite #64 exercises.

16 · The digital workforce — suite #65

> **The badge is the opt-in.** An AI worker is hired like staff: mint a login, grant the `digital-worker` bundle (console → Staff, or `integrations/hermes-worker/hire-worker.sh`). The grant sheds the walk-in customer defaults — a worker is staff. Revoke the badge to fire; the tenant's AI kill-switch stops the whole workforce mid-shift.

The job

`GET /ai/v1/workforce/tasks` — the open queue, derived LIVE from real backlogs (unassigned TMF621 tickets, unapplied cash). `POST …/tasks/{id}/claim` (15-min lease, `BSS_WORKFORCE_LEASE_SECONDS`), `…/complete` (VERIFIED: a ticket task completes only when the ticket is actually resolved), `…/escalate` (a reason required — escalations are counted, not punished). `GET …/ledger` is the shift record, including the worker's self-reported model usage (labeled: its own word — the worker brings its own LLM).

Approvals (the T3 gate)

Refunds, cease, erasure: the worker FILES (`POST …/approvals` — method + `/tmf-api/` path + body + reason); nothing executes. A human approves on the Workforce tab and the stored action runs under *the approver's own token*; refusing keeps its receipt too. Filing is `workforce:use`; deciding is `ai:use` — a worker can never approve its own ask.

The scoreboard — and the controls

Console → **Workforce**: completed / escalated / deflection, average handle time, **reopen rate** (completed tickets re-checked live — the honesty metric), the crew by name and observed type, approvals queue, shift ledger, self-reported cost (labeled), and human-minutes saved — an estimate that SAYS so, computed from operator-set baselines (`BSS_WORKFORCE_BASELINE_MINUTES_TICKET/-CASH`). The tab also holds the operator's two levers: **Crew ceiling** (governance `maxWorkers`; a new worker past it is refused at claim time — surge staffing never outgrows it) and **Hire a worker** (pick a job — care / back-office / generalist — the badge is minted and the credentials shown ONCE with the matching job card to run; activating is the runtime's act, firing is the Staff-tab revoke).

Surge staffing

The staffing signal — `staffing` in the KPIs, Prometheus gauges (`bss_workforce_backlog_depth / _active_workers / _surge`, per tenant) and `WorkforceSurgeEvent` on the boundary — tells an autoscaler when the crew is behind. Scale with KEDA (`integrations/hermes-worker/k8s-surge-example.yaml`) or the dev watcher (`surge.sh`); the ceiling caps whatever the scaler does. The BSS signals and refuses; it never spawns worker processes itself.

The Hermes package (day 1)

`integrations/hermes-worker/`: `hire-worker.sh` (the badge, shown once), `config.snippet.yaml` (Hermes `~/.hermes/config.yaml` MCP block with the tool whitelist), and SKILL.md job cards (care-triage every 15 min, cash-matching daily). Runtime-neutral by construction — any MCP-speaking agent hires into the same job, badge and audit.

genalpha-bss · sixty-five suites, eleven CTKs at zero failures · the build story is *The Honest Machine* (docs/book) · the TMF vocabulary is docs/book/tmf-reference.md